

- **Backend + Mail Infrastructure**

Postfix
Dovecot
Rspamd
PostgreSQL
Redis
Node.js API
Authentication
SMTP/IMAP integration
Mail send/receive
Domain management
Mailbox management
Alias
Basic search
Docker

Frontend + Admin

React Webmail
Login
Inbox
Mail detail
Compose
Sent/Draft/Trash/Spam
Search UI
Admin dashboard
Domain management UI
User management
Alias UI
Basic settings
API integration
Testing/support

WEEK 1 -

Day 1

Setup:

- Node.js LTS
- TypeScript
- Express/Fastify
- PostgreSQL
- Prisma/Knex
- Redis
- Environment configuration
- API structure
- Logging

Structure

backend/

```
|— src/
|   |— modules/
|   |   |— auth/
|   |   |— mail/
|   |   |— domains/
|   |   |— mailboxes/
|   |   |— aliases/
|   |— middleware/
|   |— services/
|   |— config/
|   |— app.ts
|— prisma/
```

Day 2-3

PostgreSQL

Tables:

tenants

Domains

Mailboxes

Aliases

refresh_tokens

dns_verification_logs

Mailboxes:

id

domain_id

email

password_hash

quota_bytes

used_bytes

name

role

is_active

Day 3-

Postfix

Setup:

Postfix

↓

PostgreSQL

↓

Virtual Domains

↓

Virtual Mailboxes

Implement by developer -

virtual mailbox domains

mailbox maps

alias maps

SMTP receive

SMTP send

Dovecot

Setup:

Dovecot

└─ IMAP

└─ Authentication

└─ Mailbox

└─ LMTP

Rspamd + ClamAV

Basic:

Incoming Mail

↓

Rspamd

↓

ClamAV

↓

Dovecot Mailbox

Week 2

Node.js Mail APIs

Authentication

POST /api/v1/auth/login

POST /api/v1/auth/refresh

POST /api/v1/auth/logout

Flow:

Next.js

↓

Node.js

↓

Dovecot Authentication

↓

JWT Access Token

+

Refresh Token

Mail APIs

GET /api/v1/mail/folders

GET /api/v1/mail/messages

GET /api/v1/mail/messages/:uid

POST /api/v1/mail/send

PATCH /api/v1/mail/messages/:uid/flags

POST /api/v1/mail/messages/:uid/move

DELETE /api/v1/mail/messages/:uid

GET /api/v1/mail/search

Developer 2

Next.js

Week 1- Day 1

Next.js setup:

Next.js

TypeScript

Tailwind

React Query/TanStack Query

Zod

Structure:

web/

```
└─ app/
  |   └─ login/
  |   └─ mail/
  |     |   └─ inbox/
  |     |   └─ sent/
  |     |   └─ drafts/
  |     |   └─ trash/
  |     |   └─ spam/
  |     |   └─ [id]/
  |     |
  |     └─ compose/
  |     └─ settings/
  |     └─ admin/
```

```
|
|
├── components/
├── services/
├── hooks/
└── lib/
```

Day 2 — Login

Screen:

Email

Password

[Login]

API: POST /api/v1/auth/login

Next.js ko token handling secure way

Day 3–5 — Webmail Layout



Week 2 — Next.js Mail Features

Inbox

Show:

Sender

Subject
Snippet
Date
Read/Unread
Flag

Mail Detail

From
To
CC
Date
Subject
Body
Attachments
Reply
Forward

HTML email render karte time **DOMPurify/sanitization** mandatory rakho.

Compose

To
CC
BCC
Subject
Body
[Send]

Week 3 – Admin

Developer 1 — Node.js

APIs:

POST /api/v1/admin/domains

GET /api/v1/admin/domains

GET /api/v1/admin/domains/:id/dns-check

PATCH /api/v1/admin/domains/:id
POST /api/v1/admin/mailboxes
GET /api/v1/admin/mailboxes
PATCH /api/v1/admin/mailboxes/:id
DELETE /api/v1/admin/mailboxes/:id
POST /api/v1/admin/aliases

Developer 2 - Next.js

Admin screens:

/admin
/admin/domains
/admin/users
/admin/aliases
/admin/settings

Domain screen:

Domain
Status
MX
SPF
DKIM
DMARC
Users

User screen:

Email
Name
Quota
Status
Role
Actions

Week 4 — Search + Security + Final Integration

Developer 1

Search

Install:

Meilisearch

Flow:

Email

↓

Indexer

↓

Meilisearch

↓

Node.js

↓

Next.js

Security

Implement:

- Zod validation
- rate limiting
- secure headers
- password hashing
- JWT validation
- SMTP restrictions
- Rspamd
- ClamAV

Developer 2 — Next.js Final Work

Search UI

 Search mail... |

Invoice from Rahul

Invoice July

Invoice Payment

Settings

Profile

Password

Mailbox

Quota

Session

Error Handling

Proper:

- loading
- empty state
- error state
- retry
- unauthorized
- session expired

Last 3 Days — Both Developers

Day 28

Integration testing:

Next.js

↓

Node.js

↓

Postfix/Dovecot

↓

External Gmail

Test:

Gmail → Your Mail

and

Your Mail → Gmail

and

User A → User B

Day 29

Bug fixing + security testing.

Then Next New Task -

Existing email provider se mailbox ko naye mail server me transfer karna.

Example:

Gmail

↓

Migration

↓

Your Mail Server

user@gmail.com

↓

user@company.com

Supported sources original plan me:

- Gmail
- Office 365
- cPanel
- Exchange

Infrastructure

Install:

imapsync

Redis

BullMQ

Backend

POST /api/v1/migration/jobs

GET /api/v1/migration/jobs/:id

POST /api/v1/migration/jobs/:id/pause

POST /api/v1/migration/jobs/:id/resume

POST /api/v1/migration/jobs/:id/cancel

Flow

Next.js

↓

Node API

↓

BullMQ

↓

Redis

↓

Migration Worker

↓

imapsync

↓

Dovecot

Original document exactly BullMQ + Redis + imapsync worker + WebSocket progress + resume/idempotency architecture deta hai.

Database

migration_jobs

id

mailbox_id

source_provider

source_host

source_auth_type

source_credentials_enc

status

progress_percent

last_log_line

started_at

completed_at

(Ye fields source schema me already defined hain.)

Developer split

Developer 1:

- imapsync
- worker
- BullMQ
- Redis
- migration API

Developer 2:

- migration wizard
- provider selection
- credential form
- progress bar
- logs
- pause/resume UI

Frontend

Condition ↓ Operator ↓ Value ↓ Action ↓ Folder

Backend

POST /api/v1/sieve/rules

GET /api/v1/sieve/rules

PATCH /api/v1/sieve/rules/:id

DELETE /api/v1/sieve/rules/:id

(“Backend visual JSON ko actual **Sieve script** me convert karega.”)

Database (sieve_scripts)

id

mailbox_id

name

raw_script

rule_json

is_active

updated_at

DAY 39–41 — 2FA

Technology – TOTP

Flow:

Password

↓

2FA enabled?

↓

OTP

↓

JWT

APIs

POST /auth/2fa/setup

POST /auth/2fa/verify

POST /auth/2fa/backup-codes

POST /auth/2fa/disable

(“Original document ye exact API set deta hai.”)

Database

Existing mailboxes me:

two_factor_secret

two_factor_enabled

Aur:

two_factor_backup_codes

id

mailbox_id

code_hash

used

(“Source DB schema me ye fields/tables already defined hain.”)

Frontend

Next.js:

Settings

↓

Security

↓

Enable 2FA

↓

QR Code

↓

OTP

↓

Enabled

Login:

Email

Password

↓

OTP

↓

Dashboard

DAY 42–44 — DKIM Automatic Rotation

Ye mail infrastructure ka important part hai.

DKIM kya hai?

Outgoing mail ko cryptographically sign karta hai.

Your Mail

↓

DKIM Sign

↓

Internet

↓

Receiver

↓

Public Key DNS

↓

Verify

Database

domains:

dkim_private_key

dkim_public_key

dkim_selector

Backend

POST /admin/domains/:id/dkim/rotate

Backend:

1. New key pair generate
2. Private key securely store
3. Public key admin ko show
4. DNS update instruction
5. Rspamd config update
6. Old key transition period
7. New selector activate

(“Original plan per-domain DKIM signing + automatic rotation rakhta hai.”)

Important

DNS automatically change nahi hoga unless tum DNS provider API integrate karo.

Otherwise:

System generates key



Admin copies DNS TXT



DNS provider



Verify

DAY 45–47 — S3 / MinIO Attachments

Problem

Large attachments ko mail server ke local disk par permanently rakhna ideal nahi.

Architecture

Next.js



Node.js



Presigned URL



S3 / MinIO

Upload

User selects file



Node generates presigned URL

↓

Next.js uploads directly

↓

S3/MinIO

Database

Recommended additional table:

attachments

id

mailbox_id

message_uid

object_key

file_name

mime_type

size_bytes

storage_provider

created_at

Original document S3/MinIO attachment offload aur presigned URLs specify karta hai, but separate attachments table ka detailed schema document me define nahi hai. isliye ye **implementation recommendation** hai, source-defined schema nahi.

DAY 48–49 — Undo Send

Concept

User Send karta hai:

Send

↓

BullMQ

↓

WAIT 10 seconds

↓

Postfix

Agar user:

UNDO

press kare:

Cancel Queue Job

Backend

POST /mail/send

POST /mail/undo-send

Better implementation:

POST /mail/send

↓

create delayed BullMQ job

↓

jobId

↓

wait N seconds

↓

send via Postfix

Original plan delayed queue ke through Undo Send specify karta hai.

DAY 50–51 — Schedule Send

User:

Send Later

Tomorrow

09:00 AM

Backend:
BullMQ delayed job

Database me recommended table:

scheduled_messages

id

mailbox_id

message_data

scheduled_at

status

job_id

created_at

Source document Schedule Send API aur BullMQ delayed jobs define karta hai, lekin dedicated table ka schema explicitly nahi deta.

API

POST /mail/schedule-send

DAY 52-53 — Snooze

User:

Snooze

↓

Tomorrow 9 AM

Mail inbox se temporarily hide.

BullMQ scheduled job:

Mail

↓

snooze

↓

Redis/BullMQ

↓

time reached

↓

Inbox visible

API:

POST /mail/snooze

Original plan scheduled job ke through snooze ko resurface karta hai.

DB

Recommended:

snoozed_messages

id

mailbox_id

message_uid

snooze_until

status

created_at

("Again, this table is an implementation recommendation; original schema does not define it explicitly.")

PHASE 3 — COLLABORATION

Calendar + Contacts

Calendar Backend

Technology

Radicale + CalDAV

Architecture:

Next.js Calendar



Node.js



CalDAV



Radicale



PostgreSQL

Original document Radicale + custom PostgreSQL storage backend aur JWT SSO bridge specify karta hai.

APIs

POST /calendar/events

GET /calendar/events

PATCH /calendar/events/:id

DELETE /calendar/events/:id

POST /calendar/rsvp

Features

- Create
- Edit
- Delete
- Month
- Week
- Day
- Recurring
- Attendees
- Location
- RSVP

DB

Source me:

caldav_carddav_objects

collection_id

href

data

etag

last_modified

DAY 60–64 — Calendar Frontend

Next.js:

/calendar

Views:

Month

Week

Day

Event modal:

Title

Start

End

Location

Attendees

Repeat

Email integration:

Meeting Invite

↓

.ics

↓

Email

(“Original document RFC 5545 .ics generation aur inbound .ics parsing specify karta hai.”)

DAY 65–67 — Contacts Backend

CardDAV

Next.js

↓

Node.js

↓

CardDAV

↓

Radicale

APIs:

POST /contacts

GET /contacts

PATCH /contacts/:id

DELETE /contacts/:id

GET /contacts/global

GET /contacts/suggest

(“Source plan personal/global contacts aur Redis-backed suggestions specify karta hai.”)

Database

contacts_cache

mailbox_id

email

display_name

interaction_count

last_used_at

DAY 68–70 — Contacts Frontend

Next.js: /contacts

Features:

- Personal contacts
- Global contacts
- Add
- Edit
- Delete
- Search
- Autocomplete

Compose:

To: rah...



Redis



Rahul@company.com

(“Source frontend plan recipient autocomplete specifically include karta hai.”)

PHASE 4 — BILLING

DAY 71–76

Technology

Choose:

Razorpay OR Stripe

India-focused system ke liye main Razorpay prefer karunga; international SaaS expansion ho to Stripe bhi support kar sakte ho.

Flow

Admin

↓

Choose Plan

↓

Payment Provider

↓

Webhook

↓

Node.js

↓

PostgreSQL

APIs

POST /billing/subscribe

POST /billing/webhook

(“Source plan dono endpoints aur Razorpay/Stripe webhook handling specify karta hai.”)

DB

billing_subscriptions

id

tenant_id

plan

status

current_period_end

payment_provider

provider_subscription_id

(“Source schema me exactly ye table hai.”)

Plans

Example:

Trial

Starter

Pro
Enterprise

Source schema bhi in plan categories ko indicate karta hai.

Plan Enforcement

Example:

Starter
max users = 10
Current users = 10
Create user
↓
BLOCK

(“Source Phase 5 me plan-enforcement middleware define karta hai.”)

PHASE 5 — MOBILE APP

DAY 77–90 - Technology

React Native

Backend same Node.js APIs reuse karega.

Node.js API
/ | \
/ | \
Next.js Android iOS

Source plan bhi React Native ko existing API reuse karne ke liye rakhta hai.

Features

Mobile:

- Login

- Inbox
- Mail detail
- Compose
- Reply
- Forward
- Search
- Push notification
- Contacts
- Calendar

Push

Firestore Cloud Messaging:

New Email

↓

Dovecot IMAP IDLE

↓

Node.js

↓

FCM

↓

Mobile

Source FCM project aur IMAP IDLE → FCM bridge specify karta hai.

DB

New:

devices

id

mailbox_id

device_token

platform

created_at

last_seen_at

(“Source document specifically new devices table add karne ko kehta hai.”)

PHASE 6 — KUBERNETES

DAY 91–100

Ye sabse heavy infrastructure phase hai.

Current Docker Compose:

Docker Compose

ko:

Kubernetes

me migrate karenge.

Source document bhi Compose setup ko Kubernetes manifests/Helm chart me migrate karne ko kehta hai.

Kubernetes components

Ingress

↓

Next.js

↓

Node.js

↓

Services

Mail:

Load Balancer

↓

Mail Node 1

Mail Node 2

Kubernetes objects

Deployment

Service

ConfigMap

Secret

Ingress

PVC

HPA

PHASE 7 — HA / REPLICATION

DAY 101–115

Ye **true production-grade infrastructure** hai.

Dovecot replication

Mail Node 1

↕

dsync



Mail Node 2

Source specifically Dovecot dsync replication aur multiple Dovecot nodes behind TCP load balancer specify karta hai.

PostgreSQL

PostgreSQL Primary



Streaming Replication

PostgreSQL Standby

Source PostgreSQL streaming replication primary/standby specify karta hai.

Failover

Node 1 DOWN



Load Balancer



Node 2

And DB:

Primary DOWN



Standby



Promote

(“Source failover testing hooks bhi specify karta hai.”)

SERVER REQUIREMENT BHI AB CHANGE HOGA

Phase 1

SERVER 1

Mail

SERVER 2

Next.js

Node.js

PostgreSQL

Redis

Search

Phase 2–4

Recommended:

SERVER 1

Mail

SERVER 2

Application

SERVER 3

Database

SERVER 4

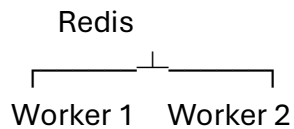
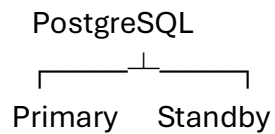
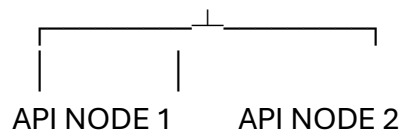
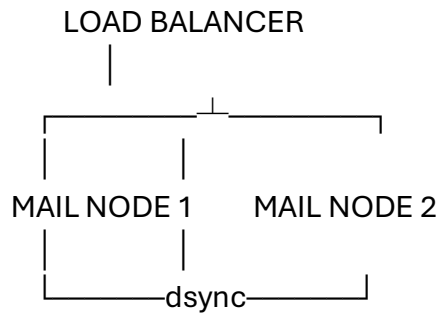
Redis/Workers

SERVER 5

Search/Object Storage

Phase 5–7

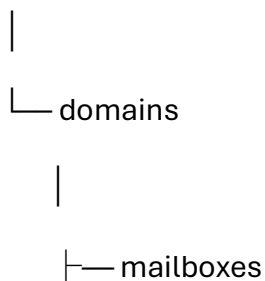
Production HA:



Source Phase 5 itself multiple Dovecot nodes, TCP load balancer, PostgreSQL primary/standby, Kubernetes aur FCM architecture define karta hai.

Database — Final Structure

tenants



- | └─ refresh_tokens
- | └─ 2FA backup codes
- | └─ sieve_scripts
- | └─ contacts_cache
- | └─ devices
- |
- └─ aliases
- └─ dns_verification_logs
- └─ rate_limit_policies

migration_jobs
caldav_carddav_objects
billing_subscriptions
webhooks
audit_logs
deletion_requests

